# SQLord: A Robust Enterprise Text-to-SQL Solution via Reverse Data Generation and Workflow Decomposition

Song Cheng*
Alibaba Group
Hangzhou, Zhejiang, China
chengsong.cs@alibaba-inc.com

Qiannan Cheng
Alibaba Group
Hangzhou, Zhejiang, China
qiannan.cqn@alibaba-inc.com

Linbo Jin
Alibaba Group
Hangzhou, Zhejiang, China
yuyi.jlb@alibaba-inc.com

Lei Yi
Alibaba Group
Hangzhou, Zhejiang, China
yilei.yi@alibaba-inc.com

Guannan Zhang
Alibaba Group
Hangzhou, Zhejiang, China
zgn138592@alibaba-inc.com

## Abstract

Transforming natural language into SQL queries (NL2SQL) is crucial for data-driven business applications. Existing frameworks, trained on open-source datasets, struggle with complex business logic and lack domain-specific data for fine-tuning. Additionally, evaluation methods often require annotated data and executable database environments, which are scarce in real-world scenarios. To address these challenges, we propose *SQLord*, an enterprise-level NL2SQL framework. First, SQLord introduces a data reverse generation approach to convert raw SQL statements into annotated data for supervised fine-tuning (SFT). Second, it proposes a decomposition method for complex queries using an automated workflow generator. Additionally, SQLord features a comprehensive GPT-Judge evaluation framework, including Execution Evaluation (EXE), Query-SQL Evaluation (QSE), and SQL-SQL Evaluation (SSE), tailored to diverse scenarios. Offline tests significantly outperform state-of-the-art baselines, and online accuracy consistently exceeds 90, highlighting SQLord's advantages and effectiveness in complex real-world scenarios. SQLord has been successfully applied across multiple scenarios on the world's largest B2B e-commerce platform.

## CCS Concepts

• **Computing methodologies** → **Natural language processing**.

## Keywords

text to sql, llm, workflow generation

**ACM Reference Format:**
Song Cheng, Qiannan Cheng, Linbo Jin, Lei Yi, and Guannan Zhang. 2025. SQLord: A Robust Enterprise Text-to-SQL Solution via Reverse Data Generation and Workflow Decomposition. In *Companion of the 16th ACM/SPEC International Conference on Performance Engineering (WWW Companion '25), April 28-May 2, 2025, Sydney, NSW, Australia.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3701716.3715541

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
*WWW Companion '25, April 28-May 2, 2025, Sydney, NSW, Australia.*
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1331-6/25/04
https://doi.org/10.1145/3701716.3715541

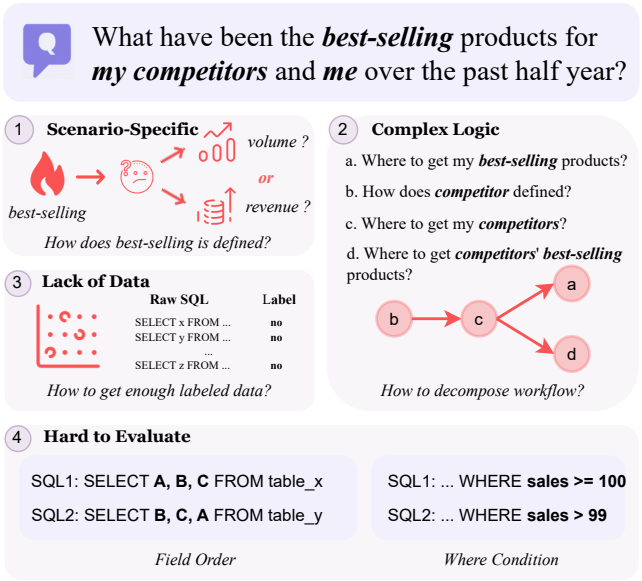



**Figure 1: A real-world online case of NL2SQL challenges**

## 1 Introduction

Nature language to SQL (NL2SQL), which translates natural language inquiries into SQL statments is still a critical challange in both natural language processing and relational database communities [9]. Historically, the researches in this domain [5–7, 15] has predominantly focused on identifying and abstracting patterns that correlate questions with SQL, subsequently refining these patterns through the deployment of encoder-decoder architectures trained on NL2SQL datasets. The emergence of large language models (LLMs) has marked a paradigm shift in methodologies for addressing the NL2SQL challenge [4, 12–14]. A notable contribution in this field was made by Gao et al. [4], who leveraged GPT-4 [11] to achieve first place on the Spider leaderboard [2], demonstrating an execution accuracy of 86.6%. Diverging from traditional approaches, the principal challenge in LLM-based NL2SQL frameworks involves

the strategic employment of prompt engineering to elicit precise SQL query generation. This process requires meticulous optimization of question representations [3, 12], careful selection of illustrative examples [8, 10], and the systematic organization of them.

Despite advancements in LLM-based methods for natural language understanding and SQL query generation, real-world enterprise applications still face significant challenges. For example, as shown in Figure 1, a user of an online B2B e-commerce platform asking about the best-selling products of both competitors and themselves over the past six months presents several difficulties. First, defining "best-selling" (e.g., by sales volume or revenue) requires domain-specific knowledge for understanding business logic, such as identifying competitors and obtaining relevant data for the task. Second, LLMs' in-context learning capabilities are often inadequate for complex NL2SQL tasks. While fine-tuning the models could improve performance, however, there is a lack of labeled SQL queries for training, as most queries in real-world applications are not labeled. Third, evaluating NL2SQL performance is challenging due to variations in SQL queries, such as differences in field order or conditions (e.g., "sales >= 100" vs. "sales > 99"), which complicate direct string matching and performance assessment, especially when queries return empty results.

To address the challenges of applying NL2SQL in real-world scenarios, we propose SQLord. First, we introduce a data reverse generation approach that leverages SQL statements developed by developers, along with their comments, to generate annotated <SQL, COMMENT> pairs. These annotated pairs are then used to train the reverse generation model, *RevLLM*, which generates natural language descriptions (referred to as "queries") for SQL statements. *RevLLM* facilitates the creation of a robust set of <Query, SQL> pairs from raw SQL commonly written in daily development, effectively mitigating the data scarcity issue for fine-tuning NL2SQL models in specialized domains. Second, we present a task decomposition strategy that combines domain-specific knowledge with automated workflow generation. This method systematically decomposes the original query into sub-tasks, enabling the handling of complex business logic, such as multi-table joins and SQL nesting, which are common in real-world applications. Third, we propose a flexible evaluation framework based on GPT-Judge, suitable for three different scenarios: EXE (Execution Evaluation) compares the execution results between SQLs, QSE (Query-SQL Evaluation) evaluates the consistency between the query and the SQL statement by leveraging a large language model (LLM), and SSE (SQL-SQL Evaluation) uses an LLM to compare the structural and semantic consistency between two SQL statements. Extensive offline experiments demonstrate SQLord's effectiveness in tackling key NL2SQL challenges. Online tests also prove the scalability and robustness of SQLord in real-time query processing. SQLord has been successfully applied on the world's largest B2B e-commerce platform, supporting essential business applications.

## 2 Framework

### 2.1 Reverse Data Generation

The scarcity of annotated <Query, SQL> pairs in specialized domains poses a significant challenge for NL2SQL model training. To address this limitation, we propose a reverse data generation approach as shown in Figure 2 (a) and (b). Specifically, we train a reverse generation model, referred to as *RevLLM*, using a small set of annotated <SQL, COMMENT> pairs $\mathcal{D}_{\text{train}} = \{(s_j, c_j) \mid j = 1, \ldots, m\}$. These pairs are collected from routine development practices, where $s_j$ represents the $j$-th SQL statement used as input and $c_j$ represents its corresponding comment as output.

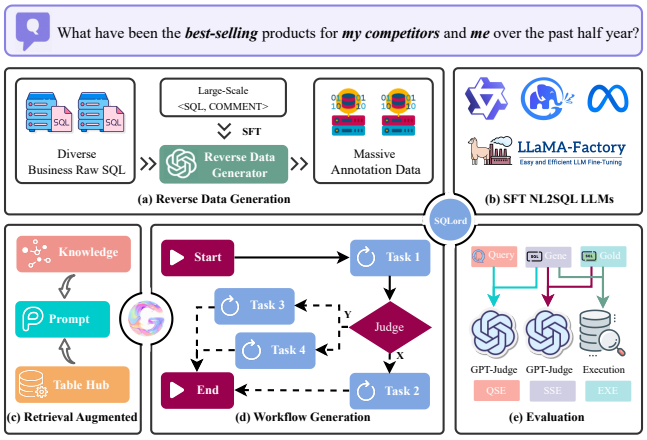



**Figure 2: The SQLord framework: (a) Reverse Data Generation for fine-tuning *SQLLM*. (b) *SQLLM* generates domain-specific SQL queries. (c) Retrieval-Augmented Generation enables task decomposition. (d) Tasks execute based on dependencies. (e) GPT-Judge evaluates SQL quality.**

The trained *RevLLM* model is then used to generate large-scale pseudo-annotated <Query, SQL> pairs for an abundant set of raw SQL statements $\mathcal{S} = \{s_1, s_2, \ldots, s_n\}$, also sourced from routine development. For each given SQL statement $s_i$, *RevLLM* generates its corresponding natural language description $q_i$, which serves as the query. This process can be formalized as:

$$q_i = \text{RevLLM}(s_i; \theta),$$

where $\theta$ represents the parameters of the model. The resulting dataset, $\mathcal{D}_{\text{gen}} = \{(q_i, s_i) \mid i = 1, \ldots, n\}$, serves as a pseudo-annotated corpus for downstream training.

Then, we fine-tune open-source LLMs, such as Qwen [1], using the generated dataset $\mathcal{D}_{\text{gen}}$. This fine-tuning process creates a domain-specific NL2SQL model, referred to as *SQLLM*, optimized for the specific business scenario. The objective of fine-tuning can be formulated as minimizing the following loss function:

$$\mathcal{L}(\phi) = -\frac{1}{|\mathcal{D}_{\text{gen}}|} \sum_{(q,s) \in \mathcal{D}_{\text{gen}}} \sum_{t=1}^{|s|} \log p(s^t \mid q, s^{<t}; \phi),$$

where $\phi$ represents the parameters of *SQLLM*, $q$ is the query, $s$ is the SQL statement, $s^t$ is the $t$-th token in $s$, and $s^{<t}$ is the prefix of tokens before $s^t$.

### 2.2 Automate Workflow Generation

The automation of complex business queries involves breaking high-level objectives into smaller, manageable sub-tasks that are executed in order of their dependencies. This can be considered as

Retrieval-Augmented Generation, leveraging domain knowledge and table schema retrieval, followed by dynamic task generation to handle the sequential and parallel nature of sub-tasks.

*Retrieval-Augmented* As shown in Figure 2 (c). Given a user query $q$, the first step is to retrieve relevant information from both the domain knowledge base $\mathcal{K}$ and the schema repository $\mathcal{T}$ (TableHub). This step provides essential context and structural information about the database, ensuring that the subsequent task decomposition aligns with both business requirements and the underlying data schema:

$$d_q = \mathrm{RA}(q; \mathcal{K}, \mathcal{T}),$$

where $\mathrm{RA}(\cdot)$ denotes a vector similarity-based retrieval system that gathers domain-specific knowledge and database schema details. The output $d_q$ represents the comprehensive context.

*Dynamic Generation* The query $q$ is progressively decomposed into sub-tasks $t_1, t_2, \ldots, t_k$ using a generator $G(\cdot)$ (an LLM, such as Qwen). Each sub-task $t_i$ depends on the previous tasks' results $\{r_1, \ldots, r_{i-1}\}$ and the information retrieved during the analysis phase. The generator $G(\cdot)$ takes into account the context $d_q$ and the intermediate results from prior tasks, ensuring that the decomposition adapts dynamically as the workflow progresses:

$$t_i = G(q, d_q, \{r_1, \ldots, r_{i-1}\}).$$

It ensures that task decomposition aligns with both the requirements of the query and the dependencies between sub-tasks.

*SQL Generation and Execution* For each generated sub-task $t_i$, a corresponding SQL query $s_i$ is produced using the trained *SQLLM*, which incorporates domain knowledge and table schema information. The generated SQL query is then executed on the database to produce the result $r_i$:

$$s_i = \mathrm{SQLLM}(t_i, d_q; \phi), \quad r_i = \mathcal{E}(s_i),$$

where $\mathcal{E}(\cdot)$ represents the SQL execution engine, and $\phi$ are the fine-tuned parameters of the model.

The tasks $t_1, t_2, \ldots, t_k$ are executed based on their dependencies, as shown in Figure 2 (d), some tasks can be executed in parallel if they are independent, while others may require sequential execution, where each subsequent task depends on the result of the previous one. This mixed execution strategy ensures efficiency and flexibility in handling the complexity of the query.

*Result* After executing all sub-tasks, the intermediate results $\{r_1, r_2, \ldots, r_k\}$ are aggregated to produce the final result $r_q$, which answers the original query $q$. The aggregation step may directly combine results or leverage an LLM to summarize and provide the final answer:

$$r_q = \mathrm{Summary}(\{r_1, \ldots, r_k\}),$$

This RAG-liked approach ensures that complex business queries are resolved systematically. By dynamically generating tasks based on retrieved knowledge, and executing them with an adaptive execution strategy, the process effectively handles intricate dependencies, making it suitable for large-scale, real-world applications.

## 2.3 Evaluation with GPT-Judge

Evaluating NL2SQL models is challenging, as existing evaluation methods often struggle to effectively adapt to diverse scenarios, as well as the semantic variability (e.g., "sales >= 100" vs. "sales > 99") and field order diversity inherent in SQL. To address this, we propose a comprehensive and flexible evaluation framework with *GPT-Judge* as shown in Figure 2 (e)., which supports multiple evaluation modes tailored to varying levels of information availability. This framework ensures robust assessment of generated SQL statments, even in incomplete settings.

*Evaluation Modes* This evaluation framework consists of three distinct evaluation modes:

1. Execution Evaluation (EXE): When a database environment is available, the generated SQL query $s_{\text{gen}}$ and the ground truth SQL query $s_{\text{gold}}$ are executed against the database to compare their results. Let $r_{\text{gen}}$ and $r_{\text{gold}}$ represent the execution results:

$$\text{EXE Score} = \mathbb{I}(r_{\text{gen}} = r_{\text{gold}}),$$

where $\mathbb{I}(\cdot)$ is an indicator function that evaluates to 1 if the results match and 0 otherwise. This approach ensures the functional equivalence of execution results, regardless of syntactic differences.

2. Query-SQL Evaluation (QSE): In scenarios where the ground truth SQL query $s_{\text{gold}}$ is unavailable, GPT-Judge leverages an LLM (e.g., GPT-4) to evaluate whether the generated SQL query $s_{\text{gen}}$ aligns with the intent of the natural language query $q$. The model considers the context retrieved via $\mathrm{RA}(\cdot)$, including schema details and domain-specific knowledge, to make an informed judgment:

$$\text{QSE Score} = \mathrm{LLMJudge}(q, s_{\text{gen}}, d_q),$$

where $\mathrm{LLMJudge}(\cdot)$ represents the evaluation function performed by the LLM, assessing semantic consistency between $q$ and $s_{\text{gen}}$.

3. SQL-SQL Evaluation (SSE): When both the generated SQL $s_{\text{gen}}$ and the ground truth SQL $s_{\text{gold}}$ are available, GPT-Judge evaluates their similarity directly. This includes comparing structural elements (e.g., SELECT fields, WHERE conditions) and overall semantics, which can be challenging due to variations in SQL expressions (e.g., "sales > 99" vs. "sales >= 100"):

$$\text{SSE Score} = \mathrm{LLMJudge}(s_{\text{gen}}, s_{\text{gold}}).$$

SQLord integrates three evaluation modes to provide a rigorous and adaptable assessment framework, bridging the gap between research methodologies and real-world applications while enhancing SQLord's robustness in complex scenarios.

# 3 Experiments

In this section, we conduct extensive offline evaluations on the open-source Spider [2] dataset and a manually annotated dataset from two enterprise scenarios, and validate SQLord online on the world's largest B2B e-commerce platform across multiple real-world application scenarios.

*Dataset* The dataset comprises 6000 `<Query, SQL>` pairs, from two online scenarios: *Customs Import-Export Assistant* for trade data analysis, and *Intelligent Product Selection* to assist merchants in product sourcing. These queries reflect the complexities of real-world

NL2SQL tasks on the world's largest B2B e-commerce platform. Additionally, *Spider* is an open, large-scale, cross-domain text-to-SQL dataset annotated by 11 Yale students.

*Implementation* It was implemented with `Qwen2-7B-Instruct` (referred to simply as `Qwen`) [1] as the base LLM. The reverse data generation model `RevLLM` was trained on 73,589 `<SQL, COMMENT>` pairs, generating 35,948 pseudo-annotated `<Query, SQL>` pairs, which were used to fine-tune `SQLLM`. They are trained in 3 epochs with a learning rate of 1e-5 and a batch size of 1. All experiments were conducted on 8 NVIDIA H100 GPUs, with evaluation based on execution accuracy (EXE), query-SQL consistency (QSE), and SQL-SQL equivalence (SSE).

**Table 1: Offline Evaluation Results on Real-World Dataset**

| Method | EXE (%) | QSE (%) | SSE (%) |
|---|---|---|---|
| Qwen | 51.3 | 58.1 | 51.7 |
| GPT-4 | 68.5 | 75.6 | 69.2 |
| DIN-SQL (GPT) | 71.4 | 78.3 | 71.9 |
| DAIL-SQL (GPT) | 72.9 | 78.7 | 72.3 |
| SQLord (Qwen) | **86.5** | **93.2** | **85.8** |

**Table 2: Offline Evaluation Results on Spider Dataset**

| Method | EXE (%) | QSE (%) | SSE (%) |
|---|---|---|---|
| Qwen | 66.7 | 72.9 | 67.1 |
| GPT-4 | 80.2 | 87.6 | 80.8 |
| DIN-SQL (GPT) | 85.3 | 91.5 | 85.9 |
| DAIL-SQL (GPT) | 86.6 | 93.4 | 86.8 |
| SQLord (Qwen) | **87.4** | **93.5** | **87.2** |

## 3.1 Offline Evaluation

We evaluate SQLord on two datasets: the Real-World Dataset, consisting of 6,000 annotated `<Query, SQL>` pairs from two enterprise scenarios, and the Spider. Evaluation metrics include execution accuracy (EXE), query-SQL consistency (QSE), and SQL-SQL equivalence (SSE). We compare SQLord with several baselines: Qwen; GPT-4 [11], using in-context learning; DIN-SQL (GPT) [12] and DAIL-SQL (GPT) [4], GPT-based frameworks; and SQLord (Qwen), with Qwen as the base LLM. Results are shown in Tables 1 and 2.

SQLord outperforms all baselines, with SQLord (Qwen) achieving the highest scores across both evaluation datasets. On the Real-World Dataset, it achieves 86.5% EXE, 13.6 percentage points higher than DAIL-SQL (GPT), and 87.4% on the Spider Dataset, showcasing its generalization ability. Additionally, the consistent differences in QSE and SSE scores (±7%) confirm GPT-Judge's reliability, with a consistent gap between EXE and GPT-Judge ( ±7% for QSE and ±1% for SSE), validating its effectiveness as an execution-independent evaluation method. These results highlight SQLord's robustness across diverse tasks.

**Table 3: Ablation Study Results on Real-World Dataset**

| Configuration | EXE (%) | QSE (%) | SSE (%) |
|---|---|---|---|
| Qwen | 51.3 | 58.1 | 51.7 |
| + Reverse Generation | 63.1 | 69.8 | 63.7 |
| + Workflow Generation | 74.6 | 81.2 | 74.3 |
| SQLord (Qwen) | **86.5** | **93.2** | **85.8** |

## 3.2 Ablation Studies

To assess the contribution of SQLord's core components, we conduct ablation studies on the Real-World Dataset using Qwen as the base LLM. The configurations tested include the baseline Qwen model, followed by the addition of reverse data generation, workflow generation, and the full SQLord framework. The results are shown in Table 3, demonstrating that each component significantly enhances SQLord's performance. Reverse data generation and workflow generation provide the largest improvements, with SQLord (Qwen) achieving an EXE score of 86.5%, a QSE score of 93.2%, and an SSE score of 85.8%, outperforming the baseline Qwen by 35.2%, 35.1%, and 34.1%, respectively.

**Table 4: Online Evaluation Results**

| Scenario | EXE (%) | QSE (%) | SSE (%) |
|---|---|---|---|
| Customs Import-Export Assistant | +15.6 | +14.4 | +16.1 |
| Intelligent Product Selection | +16.4 | +15.9 | +16.7 |

## 3.3 Online Evaluation

SQLord (Qwen) was applied in two enterprise application scenarios to evaluate the accuracy of generated SQL queries in real-world environments, using execution accuracy (EXE), query-SQL consistency (QSE), and SQL-SQL equivalence (SSE) as metrics. We compare the experimental performance before and after the launch, the results are shown in Table 4. It demonstrates SQLord's strong performance across both scenarios with a great increase in EXE scores: +15.6% for *Customs Import-Export Assistant* and +16.4% for *Intelligent Product Selection*, highlighting its robustness and adaptability in handling complex real-world queries.

## 4 Conclusion

This paper presents SQLord, an enterprise-oriented NL2SQL framework that effectively addresses domain-specific challenges through reverse data generation, workflow decomposition, and a robust evaluation framework. Experiments on real-world datasets and successful deployments in enterprise scenarios demonstrate SQLord's superiority over existing methods in handling complex business queries with high accuracy and efficiency. SQLord's modular architecture ensures adaptability to evolving business needs, while its integration with large language models highlights its potential for continuous improvement and innovation in the NL2SQL domain. By bridging research and practical applications, SQLord provides a scalable foundation for advancing intelligent data systems in diverse industries.

## References

[1] 2024. Qwen2 Technical Report. (2024).
[2] LILY Group at Yale University. 2018. Spider 1.0, Yale Semantic Parsing and Text-to-SQL Challenge. (2018). https://yale-lily.github.io/spider
[3] Xuemei Dong, Chao Zhang, Yuhang Ge, Yuren Mao, Yunjun Gao, Lu Chen, Jinshu Lin, and Dongfang Lou. 2023. C3: Zero-shot Text-to-SQL with ChatGPT. *CoRR* abs/2307.07306 (2023).
[4] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proceedings of the VLDB Endowment* 17, 5 (2024).
[5] Binyuan Hui, Ruiying Geng, Lihan Wang, Bowen Qin, Yanyang Li, Bowen Li, Jian Sun, and Yongbin Li. 2022. $S^2$SQL: Injecting Syntax to Question-Schema Interaction Graph Encoder for Text-to-SQL Parsers. In *Findings of the Association for Computational Linguistics: ACL 2022, Dublin, Ireland, May 22-27, 2022*. Association for Computational Linguistics, 1254–1262.
[6] Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. 2023. Resdsql: Decoupling schema linking and skeleton parsing for text-to-sql. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 13067–13075.
[7] Jinyang Li, Binyuan Hui, Reynold Cheng, Bowen Qin, Chenhao Ma, Nan Huo, Fei Huang, Wenyu Du, Luo Si, and Yongbin Li. 2023. Graphix-t5: Mixing pre-trained transformers with graph-aware layers for text-to-sql parsing. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 13076–13084.
[8] Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen. 2022. What Makes Good In-Context Examples for GPT-3?. In *Proceedings of Deep Learning Inside Out: The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures, DeeLIO@ACL 2022, Dublin, Ireland and Online, May 27, 2022*. Association for Computational Linguistics, 100–114.
[9] Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuyu Luo, Yuxin Zhang, Ju Fan, Guoliang Li, and Nan Tang. 2024. A Survey of NL2SQL with Large Language Models: Where are we, and where are we going? *CoRR* abs/2408.05109 (2024).
[10] Linyong Nan, Yilun Zhao, Weijin Zou, Narutatsu Ri, Jaesung Tae, Ellen Zhang, Arman Cohan, and Dragomir Radev. 2023. Enhancing text-to-SQL capabilities of large language models: A study on prompt design strategies. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 14935–14956.
[11] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023). https://doi.org/10.48550/arXiv.2303.08774
[12] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
[13] Nitarshan Rajkumar, Raymond Li, and Dzmitry Bahdanau. 2022. Evaluating the Text-to-SQL Capabilities of Large Language Models. *CoRR* abs/2204.00498 (2022).
[14] Immanuel Trummer. 2022. CodexDB: Synthesizing code for query processing from natural language instructions using GPT-3 Codex. *Proceedings of the VLDB Endowment* 15, 11, 2921–2928.
[15] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. 2020. RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*. Association for Computational Linguistics, 7567–7578.